

Canonical Dataset Lineage for Decentralized Agent Networks

Maura Clark
Contributor to the UOR Foundation
maura@uor.foundation

Abstract—Agents in decentralized networks re-export, derive, refresh, and hand off datasets. Identifiers computed over serialized bytes fork under these operations: the same logical dataset, re-encoded by a different tool, acquires a new identity, severing its lineage and freshness history. We present a protocol for canonical dataset lineage that composes existing IETF supply-chain transparency standards instead of inventing new machinery. It has four parts: a declared NFC-then-JCS canonical identity over an exact integral numeric domain; typed parent references that commit a dataset to its inputs; owner-signed publication as SCITT Signed Statements; and freshness established from Transparency Service evidence rather than writer clocks. Requirements are drawn from maintainer review of five closed contributions to NANDA Town, an open testbed for agent protocols, and each retained review vector maps to an evaluator stage. A standalone prototype of the canonicalization pipeline produces no identity forks across seven re-encodings of one dataset, where Town’s current record fingerprint forks on four, and fires every one of twenty-one pinned rejection vectors. Signing, traversal, freshness, and the in-testbed evaluation are proposed, not yet measured.

Index Terms—agent networks, provenance, content addressing, canonicalization, transparency logs, SCITT

I. Introduction

Networked agents do not just exchange messages; they pass data. A procurement agent exports a parts list; a planning agent re-imports it through a different JSON library, derives an assembly plan, and publishes that as a new dataset; an auditor later asks which inputs the plan came from and whether they were current. Each step is routine. Yet if dataset identity is a hash of the bytes each tool happened to emit, every re-encoding mints a new identifier, and lineage, freshness, and any claim bound to the old identifier silently detach. An adversary can exploit the same property deliberately: re-encoding a dataset launders it away from its history.

The relevant building blocks exist but have not been assembled for agent datasets. RFC 8785 JCS [1] fixes key order and number serialization but, by design, performs no Unicode normalization, so the composed (NFC) and decomposed (NFD) forms of the same text [10] hash differently. The IETF SCITT architecture [2] provides signed statements and append-only Transparency Services, and the Canonical Payload Binding (CPB) draft [3] defines how a payload profile declares a canonicalization, derives an identifier, and cites other artifacts by typed digest reference. What is missing is a dataset profile: an identity

TABLE I
One dataset, four encodings, measured on Town df0b5f1. Values are digest prefixes; bold marks a fork.

Encoding	fingerprint	JCS	NFC→JCS
original	965293f8	965293f8	965293f8
keys reordered	965293f8	965293f8	965293f8
1 → 1.0	6332808b	965293f8	965293f8
“Café” in NFD	30d12650	30d12650	965293f8

that survives re-encoding, parents a verifier can traverse, an owner bound by signature, and freshness that is proven rather than asserted.

This paper contributes: (i) requirements for dataset lineage taken from maintainer review of five closed contributions to NANDA Town (“Town”) [4], rather than from first principles; (ii) a protocol that meets them on the SCITT/CPB stack, including an explicitly ordered canonicalization pipeline and a precise statement of which log evidence establishes order and freshness; (iii) an evaluation design in which every retained review vector is an evaluator stage and a deliberately weak control must fail; and (iv) preliminary measurements from a standalone prototype of the canonicalization pipeline.

II. Problem and Requirements

A. Identity forks in practice

Town’s evidence layer identifies records by records.fingerprint: SHA-256 over JSON with sorted keys and compact separators [4]. As Town’s stable fingerprint of its parsed record model, which is also what its HMAC signatures cover, that is the right design. As dataset identity it is not enough. Table I encodes one logical dataset four ways. Sorted keys absorb reordering, but integer-to-float re-encoding and Unicode NFD each fork the identifier. Plain JCS repairs the number fork; only an added normalization step repairs the Unicode fork.

B. Requirements from review

Five 2026 contributions to Town were closed by the maintainers not as defects but as seeds of future profiles; each closing comment restated requirements and retained test vectors [5], [6], [7], [8], [9]. Three (#72, #50, #34) define the dataset contract, which we adopt as requirements: R1 re-encodings of the same logical

dataset resolve to one identity, and a single leaf mutation changes it; R2 empty inputs receive explicit, non-colliding handling; R3 one canonical encoding and content-derived identity, with the owner bound by signature; R4 parents are independently traversable, supported parent spellings canonicalize together, conflicting spellings fail, and a parent not bound into the canonical bytes never affects provenance; R5 freshness is verified cryptographically, not inferred from field presence, and never depends on a writer’s private clock; and R6 a dataset type carries publication, lifecycle, verification, and evaluator semantics, with a broken-link control. The other two (#59 on interview evidence, #61 on outcome-gated payments) are downstream profiles that compose on this one: a quote binds to a transcript dataset, and an evaluator criterion is committed as a dataset reference.

C. Threat model

The adversary may re-encode and republish any dataset, replay old but validly signed statements, present forged or dangling parent references, and claim another party’s ownership. It cannot forge an owner’s signature or a Transparency Service receipt. Signatures and receipts establish commitment and registration, not truth: a producer can faithfully sign a false dataset, and we do not address that.

III. Protocol

A. Canonical identity

A dataset payload has four members: content, owner, parents, and schema_version. Following CPB, the payload class declares exactly one canonicalization algorithm, CPB jcs, preceded by a profile-owned transformation. CPB requires the profile to fix the order of that transformation relative to the removal of excluded fields [3]; ours is:

- 1) Admit and validate the input bytes: strict UTF-8 without a byte-order mark, well-formed JSON, no duplicate member names after escape decoding, no non-scalar Unicode. Each number token, judged on its decimal spelling before any binary conversion, must be mathematically integral (NUMBER_NOT_INTEGRAL) and within $\pm(2^{53} - 1)$ (NUMBER_OUT_OF_RANGE).
- 2) Transform the complete payload: fold every string and member name to NFC [10]; reject names that collide after folding; enforce a closed shape (the four members, plus the legacy parent spelling and the excluded address; owner a non-empty string; schema_version "1"); normalize parent spellings and set semantics (§III-B). Validation precedes exclusion, so excluded fields cannot hide malformed input.
- 3) Exclude the top-level address member, which carries the identifier itself.
- 4) Apply RFC 8785 JCS, SHA-256, and lowercase hex.

Thus 1, 1.0, and 1e0 are one value, as R1 requires, while every non-integral quantity is carried as a string, consistent with I-JSON’s advice for exact values [11]. No absent-field

normalization is applied: {}, [], "", and null are distinct content (R2). A dataset without content is rejected. A record identified by Town’s existing fingerprint belongs to a separate artifact type, dataset-fingerprint, with its own declared digest context: the algorithm, transformation, and representation under which its digests are computed. CPB forbids comparing digests across contexts, so neither type can masquerade as the other [3].

Numeric domain as a deliberate choice. Admitting general IEEE-754 binary64 values, as plain JCS does, inherits input rounding [1]: our prototype confirmed that 0.1 and 0.10000000000000001 would share an identity. An integer-token grammar avoids that but refuses 1.0; the author’s uor-jcs-nfc context takes that route and renders digests with a sha256: prefix [15]. Admitting integral values in any spelling keeps R1’s $1 \leftrightarrow 1.0$ invariance with no aliasing; the two contexts remain distinct and non-interchangeable.

B. Typed parent references

Parents are a set of CPB typed digest references: closed objects carrying exactly type: "dataset", digest_alg: "SHA-256", and a 64-hex digest, with purpose omitted because the type has one digest context. References are carried only in the payload, never also in a cpb-refs header, and are not excluded, so a child’s identifier commits to its parents. A legacy single-parent spelling maps onto the set; identical dual spellings collapse; conflicting ones are rejected; duplicates are rejected; the set is sorted, so order never matters. Lineage is computed only from this canonical set, so a parent named anywhere else creates no edge (R4). Traversal reports each edge in a CPB reference state [3] (Verified, Failed for a recomputation mismatch, Unresolved for a broken link, Malformed) plus a profile state, Cyclic: under honest content addressing a cycle is impossible, so a revisited node signals a forged reference. No non-Verified state counts as a pass.

C. Signed publication

Publication is an RFC 9943 Signed Statement [2] (COSE_Sign1 [12]) in full-content mode, meaning the dataset itself is the signed payload, with protected content_type application/json; the CBOR Web Token issuer claim (iss) is the owner and the subject claim (sub) the derived identifier. A declared owner that differs from the issuer is rejected (R3). Verifiers report signature validity, issuer authentication, and content binding separately, and always recompute the identifier; a carried value is advisory.

D. Receipt-backed freshness

CPB content binding does not establish freshness [3], so the profile defines it on SCITT evidence and keeps three kinds of evidence separate. Inclusion: a verified receipt shows a statement was registered [13]. Order: a Transparency Service keeps registered statements in one append-only Statement Sequence [2], and only a verifiable position in it establishes supersession; for example,

RFC9162_SHA256 inclusion proofs expose a leaf index and tree size, and consistency proofs to a signed current tree head establish current state [14]. Completeness: a receipt for one statement cannot show that no later one exists, so a verifier also needs an authenticated current log state and a complete, auditable enumeration of freshness statements for the (owner, identifier) pair. The current freshness statement is the owner-authenticated one at the greatest verified position in that complete set. Replaying an old statement cannot restore freshness, non-owners cannot refresh, and producer timestamps never order anything (R5). Elapsed-time windows additionally require registration time authenticated by the Transparency Service.

IV. Town Case Study and Evaluation Design

Town is an open testbed with twelve replaceable protocol layers, logical time with no wall clock inside a run, and validators that must judge solely from an exported event log [4]. The protocol lands as an additive data-facts plugin, `dataset.v1`, beside the existing evidence plugin, whose fingerprints stay unchanged. Town’s HMAC auth plugin cannot produce third-party-verifiable statements, so an asymmetric signing plugin is required, and in Town’s local simulation mode (the Lab) an append-only log stands in for a Transparency Service.

The pilot reuses Town’s supply-chain scenario: suppliers publish part datasets, a manufacturer publishes an assembly whose parents are those parts, and an auditor re-encodes, mutates, traverses, replays, and checks freshness. A validator reports eight stages: input validation, canonical identity, mutation sensitivity, empty input, signed publication, lineage, freshness, and controls. The control plugin, `dataset-fingerprint.v1`, identifies datasets by Town’s existing fingerprint and must fail canonical identity; if it passes, the scenario is not exercising re-encoding. All sixteen retained review vectors from #72, #50, and #34 map to a stage; for example, “stale replay” is a freshness fault and “phantom parent” a lineage fault.

Preliminary results (standalone prototype, not the Town plugin). We implemented the four-step canonicalization pipeline, including shape checks and parent normalization, but not signing, traversal, or freshness, and ran it against Town’s fingerprint as the control (Table II). The pipeline produced no forks across seven re-encodings (key order, whitespace, 1.0, 1e0, NFD in a value, NFD in a member name, \u escapes); the control forked on four. Reordered parents, a legacy single parent, and identical dual spellings each resolved to one identity; a carried address was excluded and a wrong one detected; every mutation moved the identifier; the four empty values stayed distinct; and all twenty-one rejection vectors fired with their pinned reasons, nineteen distinct. The 47 vectors are published as exact input bytes with pinned outcomes, alongside the prototype and a runtime receipt [23].

TABLE II
Standalone prototype of the canonicalization pipeline versus Town’s fingerprint (control).

Check	NFC→JCS	fingerprint
Re-encodings, one identity	7/7	3/7
Parent spellings, one identity	3/3	—
Mutations, identity moved	4/4	—
Empty values, distinct ids	4/4	—
Rejection vectors fired	21/21	—

V. Related Work

SCITT [2] and Certificate Transparency [14] supply signed statements and append-only logs; CPB [3] supplies the canonicalize-derive-cite construction on which our profile is built. Software supply-chain systems such as in-toto [17] and Sigstore [18] bind attestations to artifact digests but identify artifacts by bytes, which suits build outputs and fails for structured data re-encoded in transit. Content-addressed storage [19] likewise hashes a codec’s bytes. W3C PROV [20] gives a lineage vocabulary without canonical identity or verifiable freshness. Agent protocols such as MCP locate resources by URI and signal caching with server-declared TTL hints [21]; NANDA’s registry work describes agents and their data facts [22]. Our profile supplies the missing identity, lineage, and freshness semantics these carriers could reference.

VI. Limitations and Open Questions

Numeric cost. The integral domain removes binary64 aliasing, including the float-spelled 9007199254740993.0, which an earlier token-only range check admitted; the cost is that producers must carry every fractional quantity as a string. The closed payload shape likewise pushes extensions into content or a new schema version. Completeness requires a named query, index, or monitor that no single receipt provides, and elapsed-time freshness requires service-authenticated time. Scope of evidence. Our measurements cover the canonicalization pipeline only and come from one implementation; independent reproduction, the in-testbed evaluation, and an external pilot remain future work, and a Town-authored test is not evidence of external adoption. Standards status. CPB is an individual Internet-Draft whose algorithm names are draft-local until RFC publication [3].

VII. Conclusion

Dataset identity for agent networks should survive the re-encodings agents perform, commit to its inputs, bind its owner, and prove its freshness from log evidence rather than clocks. Composing NFC-then-JCS identity, CPB typed references, and SCITT statements meets requirements that practitioners stated in review, and it is small enough to evaluate adversarially in an open testbed.

Disclosure

The author authored uor-jcs-nfc [15], a related canonicalization that this profile deliberately does not select; its specification, reference implementation, and a provisional RC5 corpus are public, and its final uor-vectors-v2 corpus is not yet designated. The author contributes to the UOR Foundation, which is not a dependency of the protocol. The closed Town contribution sic_facts by the author is cited as prior art [16].

References

- [1] A. Rundgren, B. Jordan, and S. Erdtman, “JSON Canonicalization Scheme (JCS),” RFC 8785, Jun. 2020.
- [2] H. Birkholz, A. Delignat-Lavaud, C. Fournet, Y. Deshpande, and S. Lasker, “An Architecture for Trustworthy and Transparent Digital Supply Chains,” RFC 9943, Jun. 2026.
- [3] S. Mih and A. Sokolov, “Canonical Payload Binding: A Signed Statement Construction Profile,” IETF Internet-Draft draft-mih-sokolov-scitt-payload-binding-03, Sep. 2026, work in progress.
- [4] Project NANDA, “NANDA Town,” <https://github.com/projnanda/nandatown>, commit df0b5f1fd0c2ec2f9535c786fc83694096e4e9f, Sep. 2026.
- [5] Maintainer closing review, projnanda/nandatown PR #72, 2026. <https://github.com/projnanda/nandatown/pull/72/#issuecomment-5544870992>
- [6] Maintainer closing review, PR #50, 2026. <https://github.com/projnanda/nandatown/pull/50/#issuecomment-5544906629>
- [7] Maintainer closing review, PR #34, 2026. <https://github.com/projnanda/nandatown/pull/34/#issuecomment-5544918063>
- [8] Maintainer closing review, PR #59, 2026. <https://github.com/projnanda/nandatown/pull/59/#issuecomment-5544891597>
- [9] Maintainer closing review, PR #61, 2026. <https://github.com/projnanda/nandatown/pull/61/#issuecomment-5544889492>
- [10] K. Whistler, ed., “Unicode Normalization Forms,” Unicode Standard Annex #15, Revision 53 (Unicode 15.0.0), 2022. <https://www.unicode.org/reports/tr15/tr15-53.html>
- [11] T. Bray, “The I-JSON Message Format,” RFC 7493, Mar. 2015.
- [12] J. Schaad, “CBOR Object Signing and Encryption (COSE): Structures and Process,” RFC 9052, Aug. 2022.
- [13] O. Steele, H. Birkholz, A. Delignat-Lavaud, and C. Fournet, “CBOR Object Signing and Encryption (COSE) Receipts,” RFC 9942, Jun. 2026.
- [14] B. Laurie, E. Messeri, and R. Stradling, “Certificate Transparency Version 2.0,” RFC 9162, Dec. 2021.
- [15] M. Clark, “UOR-JCS-NFC,” version 1.1 (draft), UOR Foundation repository, 2026, pinned at commit: <https://github.com/UOR-Foundation/uor-jcs-nfc/blob/303591128094362857791930fdc12b73875d1ff6/SPEC.md>
- [16] M. Clark, “sic_facts: serialization-invariant data facts,” projnanda/nandatown PR #72 (closed, unmerged), commit 4db7e43cd811d84525b3a8bae85b222d6ac89c5, 2026.
- [17] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, “in-toto: Providing farm-to-table guarantees for bits and bytes,” in Proc. USENIX Security, 2019.
- [18] Z. Newman, J. S. Meyers, and S. Torres-Arias, “Sigstore: Software signing for everybody,” in Proc. ACM CCS, 2022.
- [19] J. Benet, “IPFS – Content addressed, versioned, P2P file system,” arXiv:1407.3561, 2014.
- [20] T. Lebo, S. Sahoo, and D. McGuinness, eds., “PROV-O: The PROV Ontology,” W3C Recommendation, Apr. 2013.
- [21] Model Context Protocol, “Specification, version 2026-07-28,” <https://modelcontextprotocol.io/specification/2026-07-28>.
- [22] R. Raskar et al., “Beyond DNS: Unlocking the Internet of AI Agents via the NANDA Index and Verified AgentFacts,” arXiv:2507.14263, 2025.
- [23] M. Clark, “Canonical Dataset Lineage: Reproducibility Artifact,” GitHub repository, commit a2eef53cebcae9d89d80a58be93940780b0b4e21, Sep. 2026. <https://github.com/maurathat/canonical-dataset-lineage/tree/a2eef53cebcae9d89d80a58be93940780b0b4e21/artifact>